Specification: lta_cation_population_grouped.py

2026-07-20

Contents

1. Purpose **2**

2. Dependencies **2**

3. Constants and supported states **2**

4. Data structures **3**
- 4.1 `AtomRow` . . . 3
- 4.2 `CifData` . . . 3
- 4.3 `CandidateVariant` . . . 3
- 4.4 `PhysicalSite` . . . 3
- 4.5 `PlacedIon` . . . 3
- 4.6 Cage and chloride records . . . 4
- 4.7 `PopulationResult` . . . 4
- 4.8 `SiteSymmetryGroup` . . . 4
- 4.9 `UniqueBatchResult` . . . 4

5. Input CIF contract **5**

6. Template loading **5**

7. Variant grouping **5**

8. Allocation parsing and validation **6**
- 8.1 Species syntax . . . 6
- 8.2 Allocation resolution . . . 6
- 8.3 Capacity . . . 6
- 8.4 Auto-rebalancing . . . 6

9. Framework and chemistry validation **6**

10. Volume model **7**

11. Random population API **7**

12. Current symmetry-unique batching **8**
- 12.1 Site ordering . . . 8
- 12.2 Site symmetry group . . . 8
- 12.3 Occupation signature . . . 8
- 12.4 Batch generation . . . 9

13. Structure construction and serialization **9**

14. CLI **9**

15. Error conditions **10**

16. Current limitations **10**

17. Maximum-symmetry CLI delegation **10**

18. Test requirements **11**

18. Supporting API inventory **11**

19. Algorithmic provenance and references **12**

MS7 CLI revision **12**
v7.15 sparse chloride-occlusion extension 12

Current authority routing: supported Group-I/II species, exact integer populations, formal charges, and charge neutrality are owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 charge-equivalent dry-volume model and empirical endpoint provenance are owned by `doc/specs/v7_14_group_i_ii_volume_reference_extension.md`; the ratified sparse chloride extension is owned by `doc/specs/v7_15_chloride_occlusion_extension.md`. This document imports those semantics and does not redefine them.

1. Purpose

`lta_cation_population_grouped.py` is the core module for grouped-site Group-I/II cation population and bounded v7.15 sparse chloride-guest assembly from an LTA CIF template. It owns input parsing, symmetry expansion, grouping positional variants, role-aware allocation/charge validation, dry volume validation, random placement, canonical-template cage selection/collision checks, grouped-site dry symmetry deduplication, reports, and structure serialization.

The module does **not** perform energetic optimization, structural relaxation, or equilibrium sampling. Its CLI delegates the certified maximum-symmetry workflow to `lta_cation_maxsym_io.py`; the subgroup, optimizer, and symmetry-coupled smoothing algorithms remain outside this legacy population module.

2. Dependencies

Required runtime packages:

`Python 3`
`numpy`
`pymatgen`

Used pymatgen objects include `Lattice`, `Structure`, `SymmOp`, `Poscar`, and `LammpsData`.

3. Constants and supported states

```
SUPPORTED_CATIONS = (H, Li, Na, K, Rb, Cs, Fr, Be, Mg, Ca, Sr, Ba, Ra)
ALLOWED_ALKALI = SUPPORTED_CATIONS  # backward-compatible alias
RING_CLASSES   = (4, 6, 8)
OUTPUT_SPECIES_ORDER = (Si, Al, O, H, Li, Na, K, Rb, Cs, Fr, Be, Mg, Ca, Sr, Ba, Ra, Cl)
```

The dry framework model supports only fully occupied Al, Si, and O framework atoms with Group I (+1) and Group II (+2) compensation. Production generation is restricted to Al24Si24O96.

4. Data structures

4.1 `AtomRow`

```
AtomRow(
    label: str,
    element: str,
    frac: numpy.ndarray,
    occ: float = 1.0,
)
```

Represents one parsed or expanded atom-site record.

4.2 `CifData`

```
CifData(
    lattice: pymatgen.core.Lattice,
    symops: list[pymatgen.core.operations.SymmOp],
    atoms: list[AtomRow],
)
```

Minimal parsed CIF content.

4.3 `CandidateVariant`

```
CandidateVariant(
    frac: numpy.ndarray,
    site_label: str,
    ring_class: str,
    site_id: str = "",
    variant_id: int = 0,
)
```

A raw or grouped alternative cation coordinate.

4.4 `PhysicalSite`

```
PhysicalSite(
    site_id: str,
    ring_class: str,
    site_label: str,
    variants: list[CandidateVariant],
)
```

One physical occupation site. `n_variants` is the number of alternative coordinates. `choose_variant(rng)` samples one variant uniformly.

4.5 `PlacedIon`

```
PlacedIon(
    element: str,
    frac: numpy.ndarray,
    site_label: str,
    ring_class: str,
    site_id: str,
    variant_id: int,
    role: str = "framework",
)
```

One selected ion and its physical-site provenance.

4.6 Cage and chloride records

`CageSite` is the immutable authoritative definition of a supported alpha/beta cage center. `OccludedChloride` records the selected cage ID/type and fractional coordinate. v7.15 binds exactly two alpha and two beta centers for the canonical supported primitive setting.

4.7 `PopulationResult`

Contains:

- final `Structure`;
- expanded framework atoms;
- grouped physical sites;
- role-tagged placed cations;
- framework allocations and separately stored occlusion allocations;
- ignored template atoms;
- seed and grouping cutoffs;
- framework counts and legacy framework-only `alkali_counts`;
- derived framework-only and total cation-count views;
- occlusion-counterion counts and selected chloride cage records;
- independent framework/occlusion charge views;
- final chloride collision-distance metadata;
- input, reference, target, and written volume metadata.

Methods:

```
write(filename, significant_figures=8) -> str
write_cif(filename, significant_figures=8) -> None
write_report(filename) -> None
```

`write()` returns the detected format name.

4.8 `SiteSymmetryGroup`

```
SiteSymmetryGroup(
    site_ids: tuple[str, ...],
    ring_classes: tuple[str, ...],
    permutations: tuple[tuple[int, ...], ...],
    source_operation_count: int,
    mapping_tolerance_ang: float,
)
```

`permutations[g][i]` is the destination index of old site `i`. The current object stores only grouped-site permutations. It omits affine operations and coordinate variants.

4.9 `UniqueBatchResult`

```
UniqueBatchResult(
    members: list[PopulationResult],
    canonical_signatures: list[tuple[str, ...]],
    symmetry_group: SiteSymmetryGroup,
    attempts: int,
    rejected_equivalent: int,
)
```

5. Input CIF contract

The minimal reader requires:

- `_cell_length_a`, `_cell_length_b`, `_cell_length_c`;
- `_cell_angle_alpha`, `_cell_angle_beta`, `_cell_angle_gamma`;
- an atom-site loop with label and fractional coordinates;
- optional atom type and occupancy columns;
- optional symmetry-operation loop.

Accepted symmetry tags are:

```
_space_group_symop_operation_xyz
_symmetry_equiv_pos_as_xyz
```

Identity is used when no symmetry operation is found.

The parser supports quoted tokens and standard uncertainty notation such as `12.3(2)`. It is not a complete CIF grammar and does not support all multiline or semicolon-delimited CIF constructs.

6. Template loading

```
load_lta_template(
    input_cif,
    *,
    site_map="NA1:6,NA2:8,NA3:4",
    template_elements="Na",
    dedup_digits=8,
    variant_group_cutoffs=None,
    group_by_label=True,
) -> tuple[Lattice, list[AtomRow], dict[str, list[PhysicalSite]], list[AtomRow]]
```

Procedure:

1. parse the CIF;
2. split atoms into cation templates and framework templates by element;
3. expand both subsets under all declared symmetry operations;
4. deduplicate expanded rows by element, label, and rounded fractional coordinate;
5. classify expanded template atoms by label;
6. group candidate variants into physical sites;
7. return lattice, framework, grouped sites, and ignored template rows.

7. Variant grouping

```
group_candidate_variants(
    variants_by_ring,
    lattice,
    *,
    cutoffs,
    group_by_label=True,
) -> dict[str, list[PhysicalSite]]
```

For each ring family, construct a graph whose vertices are candidate variants. Add an edge when the minimum-image distance is no larger than the family cutoff. When `group_by_label=True`, different labels cannot be connected.

Connected components are computed with a disjoint-set forest. The data structure follows Tarjan's set-union framework; the LTA grouping rule is package-specific [4].

Physical-site IDs have the form:

`{ring}R_{label}_{index:04d}`

The grouping result must be reviewed through the capacity report because cutoff choice determines the number of physical sites.

8. Allocation parsing and validation

8.1 Species syntax

Exact counts:

`Li:4,Na:8,K:0`

Ratios:

`Li:0.25,Na:0.50,K:0.25`

Every species in `SUPPORTED_CATIONS` is accepted. Counts must be nonnegative integers; noninteger numeric values are invalid rather than truncated. Ratios must lie in $[0, 1]$.

8.2 Allocation resolution

```
allocation_from_count_or_ratio(
    *,
    n_avail,
    count_spec,
    ratio_spec,
    ring_class,
) -> dict[str, int]
```

Counts and ratios are mutually exclusive within one ring family. Ratios are converted by independent rounding:

$$N_\alpha = \mathrm{round}(x_\alpha n_{\mathrm{avail}}).$$

The sum is not automatically normalized to capacity.

8.3 Capacity

`validate_allocations_against_capacity(physical_sites_by_ring, allocations)`

Requires

$$\sum_\alpha N_{r,\alpha} \le |\mathcal{P}_r|.$$

8.4 Auto-rebalancing

`rebalance_allocations_to_capacity(...)`

Moves overflow one ion at a time to an available family in the fixed species-preference order. It preserves global species counts but changes ring-family counts. It is used only when explicitly enabled.

9. Framework and chemistry validation

`validate_framework_model(framework_atoms) -> None`

Requirements:

- unit occupancy within tolerance;
- no supported Group I/II cation in the framework subset;

- framework elements limited to Al, Si, and O.

The user-facing generation boundary additionally requires the supported production framework `Al24Si24O96` (Si:Al = 1:1). Non-24-Al LTA cells and other Si:Al ratios are deferred to a future scientific/numerical implementation. Small non-24-Al CIFs under `tests/` are internal algorithm fixtures only.

`enforce_charge_neutrality(framework_counts, alkali_counts) -> int`

Requires

$$\sum_i z_i N_i = N_{\mathrm{Al}}.$$

Returns the required positive charge N_{Al} (retaining the historical field/API name where applicable).

10. Volume model

```
interpolated_dry_target_volume(
    alkali_counts,
    *,
    volume_factor=1.0,
) -> tuple[float, float]
```

Returns `(reference_volume, target_volume)` in Angstrom^3. v7.14 generalizes the established affine interpolation to formal-charge equivalents:

$$V_{\mathrm{ref}} = \frac{1}{24} \sum_i z_i N_i^{\mathrm{fw}} V_i^{(24)}.$$

For the supported Al24Si24O96 cell, the framework-compensating population independently satisfies `sum_i z_i N_i^fw = 24`, so the dry-model charge-equivalent weights sum to one. Occlusion-counterion cations and Cl guests are excluded from this interpolation. For Group-I-only dry framework compositions this is exactly the historical count-over-24 formula. `V_i^(24)` values are literature-derived measured or affine-extrapolated dry-LTA corners where a species-specific reference is bound. `H`, `Be`, `Fr`, and `Ra` use the standard Na-A corner as explicit placeholders because v7.14 binds no species-specific dry-LTA volume reference for them; this does not claim exhaustive absence from the literature. The input template volume is not used as a missing-species fallback. See `v7_14_group_i_ii_volume_reference_extension.md` for the source table and caveats.

The function rejects nonpositive/nonfinite volume factors and unsupported occupied species.

`isotropically_rescaled_lattice(lattice, target_volume) -> tuple[Lattice, float]`

Returns the rescaled lattice and linear scale

$$\lambda = (V_{\mathrm{target}}/V_{\mathrm{input}})^{1/3}.$$

11. Random population API

```
populate_physical_sites(
    physical_sites_by_ring,
    allocations,
    *,
    rng,
) -> list[PlacedIon]
```

Within each ring family, the function samples physical sites without replacement by species in deterministic `SUPPORTED_CATIONS` order. One variant is selected uniformly from each occupied physical site.

`generate_lta_cation_structure(..., allocations, ...) -> PopulationResult`

Uses already-resolved integer allocations.

`generate_lta_cation_structure_from_specs(...) -> PopulationResult`

Loads the template, resolves CLI-like count or ratio strings, validates chemistry, populates sites, rescales the lattice, and builds the final structure.

12. Current symmetry-unique batching

12.1 Site ordering

`ordered_physical_sites(physical_sites_by_ring)`

Returns all 4R sites, then 6R sites, then 8R sites, retaining each family's internal order.

12.2 Site symmetry group

```
build_site_symmetry_group(
    lattice,
    physical_sites_by_ring,
    symops,
    *,
    mapping_tolerance_ang=0.05,
    close_group=True,
    max_group_order=4096,
) -> SiteSymmetryGroup
```

Current algorithm:

1. compute a periodic centroid for each physical site;
2. transform each centroid by each CIF operation;
3. match to one unique site in the same ring family;
4. require a bijection;
5. append identity;
6. deduplicate induced permutations;
7. optionally close the permutation generators under composition.

Failure to map uniquely raises `ValueError`.

The closure algorithm is a direct small-group implementation with general background from Seress [3].

12.3 Occupation signature

`site_occupation_signature(...) -> tuple[str, ...]`

Each ordered site is encoded as `empty` or one member of `SUPPORTED_CATIONS`. More than one ion on the same site is an error.

`transform_site_signature(signature, permutation) -> tuple[str, ...]`

Applies an old-site-to-new-site destination map.

`canonical_site_occupation_signature(...) -> tuple[str, ...]`

Returns

$$C(s) = \min_{g \in G_P} \text{tuple}(g \cdot s).$$

Off-center variants are deliberately ignored. The broader symmetry-reduction principle follows derivative-structure and symmetry-adapted occupancy methods [1,2,5]; the fixed-cell tuple implementation is package-specific.

12.4 Batch generation

```
generate_unique_lta_cation_batch_from_specs(
    input_cif,
    *,
    batch_size,
    ...,
    max_attempts=None,
    symmetry_mapping_tolerance_ang=0.05,
) -> UniqueBatchResult
```

Candidate member seeds are drawn from a master RNG. A candidate is accepted only when its canonical signature has not appeared. Failure to reach `batch_size` before `max_attempts` raises `RuntimeError`; duplicates are never emitted to fill the request.

13. Structure construction and serialization

`build_p1_structure(lattice, framework_atoms, placed_ions, chlorides=()) -> Structure`

Framework atoms are assembled first, followed by all grouped-site cations and then selected chloride cage guests. Placed cations are sorted by ring family, element, site ID, and variant ID before the chloride records are appended.

`group_structure_by_element_for_output(structure) -> Structure`

Creates a serialization-only copy with contiguous element blocks. Supported elements use `OUTPUT_SPECIES_ORDER` (`Si, Al, O`, all supported Group-I/II cations, then `Cl`); other elements follow in first-seen order.

```
detect_output_format(filename) -> str
write_structure_by_suffix(structure, filename, *, significant_figures=8) -> str
```

Supported outputs:

Name	Writer	Notes
`*.cif`	`CifWriter`	explicit P1 structure
`*.POSCAR`, `POSCAR`, `*.vasp`	`Poscar`	one species block per element
`*.data`	`LammpsData.from_structure(..., atom_style="atomic")`	no charge column; mass comments added

`format_batch_output_path()` inserts a deterministic one-based index before the suffix. A bare `POSCAR` becomes `POSCAR_001.vasp`.

14. CLI

`python lta_cation_population_grouped.py INPUT_CIF OUTPUT_FILE [options]`

Main option groups:

- framework population: `--n4`, `--n6`, `--n8`, `--ratio4`, `--ratio6`, `--ratio8`;
- sparse occlusion: `--alpha-cl`, `--beta-cl`, `--occlusion-n4`, `--occlusion-n6`, `--occlusion-n8`, `--occlusion-min-distance`;
- template: `--site-map`, `--template-elements`, `--dedup-digits`;
- grouping: `--variant-group-cutoff`, `--variant-group-cutoff4`, `--variant-group-cutoff6`, `--variant-group-cutof
  `--no-group-by-label`;

- chemistry: `--auto-rebalance`, `--volume-factor`;
- reproducibility: `--seed`;
- batching: `--unique-batch-size`, `--unique-batch-max-attempts`, `--symmetry-map-tolerance`;
- outputs: `--report`, `--write-candidate`.

`--write-candidate-cif` is retained as a hidden compatibility alias for `--write-candidate`.

In v7.15, any nonzero occlusion request is rejected with ratio allocation, auto-rebalance, maximum-symmetry mode, symmetry-unique batch size greater than one, or candidate-variant output. These guards execute at the public dispatch boundary before downstream certificates or diagnostic artifacts are created. The destination format is still inferred from the supplied filename; the alias name does not force CIF output.

Version 7.9 adds `--population-mode random | maximum-symmetry`, with `random` as the backward-compatible default. Maximum-symmetry mode delegates to `lta_cation_maxsym_io.py` and adds symmetry backend, realization, variant-library, bounded tie, search-budget, physical-filter, precision, and post-write verification options. Legacy `--seed`, `--unique-batch-*`, and `--write-candidate` options are rejected in maximum-symmetry mode.

15. Error conditions

The module raises on:

- malformed or unsupported CIF content required by the minimal parser;
- invalid site maps, species, counts, or ratios;
- simultaneous count and ratio input for one family;
- over-capacity allocations without successful explicit rebalancing;
- invalid framework elements or occupancy;
- residual alkali in the framework;
- missing framework Al;
- non-neutral total cation count;
- invalid input or target volume;
- unsupported output filename;
- non-bijective or ambiguous grouped-site symmetry mapping;
- symmetry-group closure exceeding its guard;
- impossible requested unique batch within the attempt limit.

16. Current limitations

- CIF parsing is intentionally minimal.
- Parent symmetry is read from the CIF and is not inferred from P1 input.
- Symmetry mapping uses physical-site centroids, not full variant sets.
- `SiteSymmetryGroup.order` is the grouped-site permutation-image order, not necessarily the affine operation count.
- Unique batching measures grouped occupation only and ignores variant coordinates.
- Random variant choice can break the coordinate symmetry implied by an occupation.
- Maximum-symmetry generation and verified serialization are implemented through the dedicated MS0-MS4 modules. MS5 symmetry-coupled smoothing is implemented in `lta_cation_symmetry_smoothing.py` and remains isolated from this population module; the legacy random smoothing path still moves individual ions.

17. Maximum-symmetry CLI delegation

The main parser exposes the MS4 surface documented in `lta_cation_maxsym_io_spec.md`. It constructs a `SymmetryTolerancePolicy`, calls `generate_maximum_symmetry_lta_batch_from_specs()`, writes through `write_maximum_symmetry_batch()`, and appends post-write verification to the report. The legacy random branch remains unchanged.

18. Test requirements

Current focused tests must continue to verify:

- rotated occupations have equal canonical signatures;
- non-equivalent occupations remain distinct;
- species identity is preserved;
- variant choice does not affect grouped occupation signatures;
- generator closure constructs the expected finite group;
- batch results have pairwise distinct canonical signatures;
- impossible batches fail explicitly.

Future maximum-symmetry tests are specified in the architecture manual.

18. Supporting API inventory

The following supporting calls are part of the current module and are used by the main workflows or external scripts. They are listed explicitly so the specification covers the complete implemented surface.

Call	Input / output contract
`normalize_element(elem: str) -> str`	Normalize an element symbol; reject empty input.
`read_cif_minimal(filename) -> CifData`	Parse the restricted CIF subset defined in Section 5.
`wrap_frac(frac) -> ndarray`	Wrap each fractional component to $[0, 1)$.
`frac_key(frac, ndigits=8) -> tuple[float,float,float]`	Create a wrapped rounded coordinate key and normalize values numerically equal to 0 or 1.
`expand_atom_rows(atoms, symops, *, dedup_digits=8) -> list[AtomRow]`	Expand rows under all operations and deduplicate by element, label, and coordinate key.
`parse_site_map(text) -> dict[str,str]`	Parse `LABEL:RING` entries and require ring class 4, 6, or 8.
`classify_template_site(label, site_map) -> Optional[str]`	Match an exact label or label prefix to a ring family.
`parse_cutoffs(default, c4, c6, c8) -> dict[str,float]`	Resolve one default and optional per-family grouping cutoffs.
`UnionFind(n)`	Internal disjoint-set forest used to form connected components.
`periodic_distance(lattice, f1, f2) -> float`	Return pymatgen minimum-image distance in Angstrom.
`parse_species_spec(text, *, as_ratio) -> dict[str,float|int]`	Parse supported Group I/II cation values and validate range/type.
`format_capacity_summary(physical_sites_by_ring, allocations=None) -> str`	Produce a diagnostic site/variant capacity summary.
`make_allocations_from_cli(...) -> dict[str,dict[str,int]]`	Resolve all three ring-family count/ratio specifications and optional rebalancing.
`count_by_species(placed) -> dict[str,int]`	Count all supported placed cations, including zero entries.
`sum_allocations(allocations) -> dict[str,int]`	Sum per-family allocations into global species counts.
`count_framework_elements(framework_atoms) -> Counter[str]`	Count expanded framework elements.
`symmetry_equivalent_populations(first, second, symmetry_group) -> bool`	Compare two occupation signatures through canonicalization.
`make_unique_batch_report(batch) -> str`	Format batch-level symmetry audit text.
`make_population_report(result) -> str`	Format chemistry, volume, capacity, and placement audit text.
`build_candidate_variant_structure(lattice, framework_atoms, physical_sites_by_ring) -> Structure`	Build a visualization structure containing every candidate variant as Na; not a physical neutral output.

Internal helpers `_fcc_primitive_volume_from_conventional_a`, `_annotate_lammps_mass_symbols`, `_strip_comment`, `_tokens`, `_as_float`, `_build_population_result`, `_periodic_site_centroid`, `_compose_site_permutatio` and `_close_site_permutation_group` are implementation details. Their behavior is covered by the volume, writer, parser, result-construction, and symmetry sections above.

19. Algorithmic provenance and references

1. G. L. W. Hart and R. W. Forcade, "Algorithm for generating derivative structures," *Physical Review B* **77**, 224115 (2008). DOI: 10.1103/PhysRevB.77.224115.
2. R. Grau-Crespo, S. Hamad, C. R. A. Catlow, and N. H. de Leeuw, "Symmetry-adapted configurational modelling of fractional site occupancy in solids," *Journal of Physics: Condensed Matter* **19**, 256201 (2007). DOI: 10.1088/0953-8984/19/25/256201.
3. A. Seress, *Permutation Group Algorithms*, Cambridge University Press (2003).
4. R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," *Journal of the ACM* **22**, 215-225 (1975). DOI: 10.1145/321879.321884.
5. J.-C. Lian et al., "Algorithm for generating irreducible site-occupancy configurations," *Physical Review B* **102**, 134209 (2020). DOI: 10.1103/PhysRevB.102.134209.
6. T. Carey et al., "Chemical Control of Thermal Expansion in Cation-Exchanged Zeolite A," *Chemistry of Materials* **26**, 1561-1566 (2014). DOI: 10.1021/cm403312q.
7. T. Carey, PhD thesis, University of Birmingham (2013), Chapter 3.
8. S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, and G. Ceder, "Python Materials Genomics (pymatgen): A robust, open-source Python library for materials analysis," *Computational Materials Science* **68**, 314-319 (2013). DOI: 10.1016/j.commatsci.2012.10.028. API documentation: https://pymatgen.org/.
9. A. P. Thompson *et al.*, "LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," *Computer Physics Communications* **271**, 108171 (2022). DOI: 10.1016/j.cpc.2021.108171. Command documentation: https://docs.lammps.org/.
10. C. R. Harris *et al.*, "Array programming with NumPy," *Nature* **585**, 357-362 (2020). DOI: 10.1038/s41586-020-2649-2.

MS7 CLI revision

Maximum-symmetry mode accepts `--max-direct-occupation-count N` with default `100000`; zero disables direct raw-occupation enumeration. Existing random and unique-batch modes ignore this option.

v7.15 sparse chloride-occlusion extension

Random population now supports the bounded role-aware occlusion contract in `v7_15_chloride_occlusion_extension.md`. `PlacedIon.role` is `framework` by default for compatibility or `occlusion_counterion`. `PopulationResult.alkali_counts` and `framework_cation_counts` remain framework-only. `cation_counts` is the derived total physical cation view across both roles; `total_cation_counts` is an explicit alias for the same total view. Occluded chlorides are authoritative cage records and are appended to the single assembled structure only after the independent charge checks and collision guard pass.

Framework placement uses the legacy RNG stream first. Excess-role placement uses a deterministic secondary stream and only still-empty grouped sites. Combined ring-family occupancy may not exceed physical capacity. The dry-volume interpolation receives framework-role counts only.

Occlusion mode is fail-closed to the bound canonical primitive template and its validated alpha/beta center orbits. One requested cage in a two-cage family is selected by maximum minimum periodic distance to final cations, with canonical cage-ID tie breaking. Cl is serialized after all supported Group-I/II cations.